

Model for Math Problems Difficulty Prediction

Machine Learning Project Report

Matus Bucher

bucher5@uniba.sk

Comenius University in Bratislava

GitHub: github.com/matusbucher/ml-project

1 Introduction

We implement and evaluate models for predicting the difficulty of mathematical problems from their textual descriptions. We compare approaches spanning classical text-based models (TF-IDF), interpretable feature engineering, and fine-tuned transformers. Practical applications include exam assessment, adaptive learning, and benchmarking reasoning models. We use Numpy [1], Scikit-learn [2], Textstat [3], PyTorch [4], and Transformers [5]. A full version of this report with detailed methodology, hyperparameter search, and combined-dataset experiments is available in the project repository: github.com/matusbucher/ml-project.

2 Datasets and Problem Setup

We use two datasets with explicit difficulty annotations:

- **MATH-lighteval** [6]: 12,500 problems; difficulty as integer level 1–5.
- **Easy2Hard-Bench (E2H)** [7]: 3,980 problems; difficulty as float rating in $(0, 1)$.

Problem statements are given as text with $\text{\TeX}$ notation. We use an 80:20 train-test split (MATH: 9,998/2,500; E2H: 3,180/795). The model outputs a single value in $[0, 1]$. MATH-lighteval levels are normalized via $0.1 + 0.2(l - 1)$; E2H ratings are used as-is. Predictions are clipped to $[0, 1]$ at evaluation time. We report R^2 and RMSE.

3 Baseline Models

Four baselines establish reference performance: **Random** (uniform sample in $(0, 1)$), **Average** (training mean), **Desc. Length** (linear regression on description length), and **Sol. Length** (linear regression on solution length). Test results are in table 1.

Table 1: Baseline test-set scores

Model	MATH		E2H	
	R^2	RMSE	R^2	RMSE
Random	−0.98	0.454	−1.43	0.382
Average	0.000	0.323	−0.002	0.245
Desc. Length	0.061	0.313	−0.005	0.246
Sol. Length	0.210	0.287	0.124	0.229

4 TF-IDF with Linear Regression

We vectorize problem texts using `TfidfVectorizer` with fixed settings (`ngram_range=(1,2)`, `sublinear_tf=True`, `stop_words="english"`) and tuned settings (`min_df=3`, `max_df=0.3`). We train either *Ridge* ($\alpha = 1.0$) or *SVR* ($C = 0.1$, $\varepsilon = 0.1$), both tuned via 5-fold cross-validated grid search (negative MSE). We test three preprocessing strategies: (a) raw text, (b) $\text{\TeX}$ fully stripped, (c) only $\text{\TeX}$ punctuation removed. Results are in table 2.

Table 2: TF-IDF test-set scores

Model	MATH		E2H	
	R^2	RMSE	R^2	RMSE
Ridge (a)	0.328	0.212	0.609	0.153
Ridge (b)	0.298	0.216	0.545	0.165
Ridge (c)	0.328	0.212	0.603	0.154
SVR (a)	0.326	0.212	0.595	0.156
SVR (b)	0.294	0.217	0.526	0.169
SVR (c)	0.325	0.212	0.592	0.156

Approaches (a) and (c) perform similarly and both outperform (b), indicating mathematical notation carries useful signal. The best overall model is **TF-IDF + Ridge (a)**.

5 Linguistic Features with Tree-Based Models

We extract 11 features using Textstat [3] and custom code: sentence count, lexicon count, syllable count, Flesch reading ease, Flesch-Kincaid grade level, Dale-Chall score, difficult words count, math expression count, numbers count, operators count, and variables count. We fit a *Random*

Forest or Gradient Boosting regressor, tuned via 5-fold grid search. Best Gradient Boosting hyperparameters on MATH: `n_estimators=300`, `lr=0.01`, `max_depth=10`, `subsample=0.6`; on E2H: `n_estimators=700`, `lr=0.01`, `max_depth=5`, `subsample=0.8`. Results are in table 3.

Table 3: Linguistic feature model test-set scores

Model	MATH		E2H	
	R^2	RMSE	R^2	RMSE
Random Forest	0.295	0.217	0.444	0.183
Grad. Boosting	0.297	0.217	0.457	0.180

Both models perform comparably; Gradient Boosting generalizes slightly better. The most predictive features across both datasets (by average importance) are variables count, Dale-Chall score, and syllable count.

6 Transformer Models

We fine-tune **BERT-base** [8] and **MathBERT** [9] with a single-output regression head via the Transformers library [5]. Inputs are tokenized with `max_length=256`, padding, and truncation.

1st attempt (90:10 train-val split): `lr=2e-5`, `weight_decay=0.01`, `epochs=4`. Both models showed overfitting (train scores significantly better than test).

2nd attempt (80:20 train-val split) targeting reduced overfitting: `lr=1e-5`, `weight_decay=0.1`, hidden/attention dropout 0.2. Results are in table 4.

Table 4: Transformer model test-set scores

Model	MATH		E2H	
	R^2	RMSE	R^2	RMSE
BERT-base (1st)	0.404	0.199	0.738	0.125
MathBERT (1st)	0.383	0.203	0.701	0.134
BERT-base (2nd)	0.352	0.208	0.698	0.135
MathBERT (2nd)	0.351	0.208	0.613	0.152

The 2nd attempt reduces overfitting but at the cost of test performance. BERT-base consistently outperforms MathBERT, likely due to domain mismatch of MathBERT pre-training data.

7 Overall Comparison

Table 5 compares the best representative from each category (Description Length baseline included as the fairest baseline, since all main models use only the problem statement).

Table 5: Best model per category, test-set scores

Model	MATH		E2H	
	R^2	RMSE	R^2	RMSE
Desc. Length	0.061	0.313	-0.005	0.246
TF-IDF + Ridge (a)	0.328	0.212	0.609	0.153
LingFeat + GBR	0.297	0.217	0.457	0.180
BERT-base (1st)	0.404	0.199	0.738	0.125

8 Conclusion

BERT-base achieves the best test performance, followed by TF-IDF + Ridge. Linguistic feature models trail slightly. Retaining mathematical notation consistently improves TF-IDF results. Domain-specific pretraining (MathBERT) did not improve over general BERT. All models provide meaningful signal over naive baselines, showing that textual features alone can predict problem difficulty to a reasonable degree.

References

- [1] Charles R. Harris et al. Numpy. <https://numpy.org>.
- [2] Fabian Pedregosa et al. scikit-learn: Machine learning in python. <https://scikit-learn.org>.
- [3] Shivam B. and contributors. textstat: Readability and text statistics for python. <https://github.com/textstat/textstat>.
- [4] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch. <https://github.com/pytorch/pytorch>.
- [5] Thomas Wolf et al. Transformers: State-of-the-art natural language processing. <https://github.com/huggingface/transformers>.
- [6] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [7] Mucong Ding, Chenghao Deng, Jocelyn Choo, Zichu Wu, Aakriti Agrawal, Avi Schwarzschild, Tianyi Zhou, Tom Goldstein, John Langford, Anima Anandkumar, et al. Easy2hard-bench: Standardized difficulty labels for profiling llm performance and generalization. *arXiv preprint arXiv:2409.18433*, 2024.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. URL <http://arxiv.org/abs/1810.04805>.
- [9] tbs17. tbs17/mathbert. <https://huggingface.co/tbs17/MathBERT>.